

Sistema Integrado de Inteligencia de Negocio Crediticio para el Control de la Mora

Anteproyecto de Grado

Omar Velez

omar.velez@yachaytech.edu.ec

Universidad Yachay Tech

Maestria en Inteligencia Artificial

Julio 2026

Problema y Motivacion

- El sector de **economia social y solidaria** enfrenta altos niveles de mora crediticia
- No existen sistemas integrados que combinen:
 - Analisis historico de credito
 - Prediccion automatizada con IA
 - Visualizacion interactiva para toma de decisiones
- Procesos manuales, lentos y propensos a errores

Objetivo: Crear un pipeline end-to-end que transforme datos crediticios crudos en predicciones accionables y dashboards para el control de la mora.

Objetivos

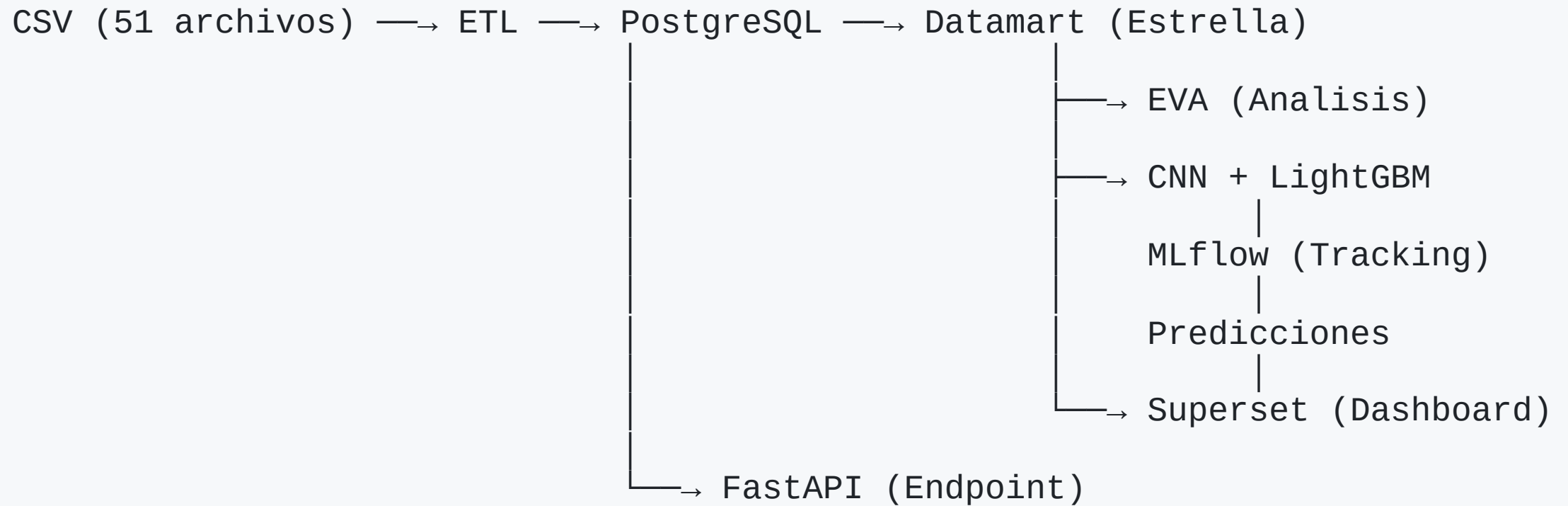
Objetivo General

Disenar e implementar un sistema integrado de inteligencia de negocio crediticio para el control de la mora en el sector de economia social y solidaria.

Objetivos Especificos

1. **Pipeline ETL** automatizado con Apache Airflow
2. **Datamart analitico** con esquema estrella en PostgreSQL
3. **Modelos de IA** (CNN + LightGBM) para prediccion multi-horizonte
4. **Dashboards interactivos** con Apache Superset
5. **Aprobacion humana** del modelo antes de produccion

Arquitectura General



Stack tecnologico: Airflow, PostgreSQL, MLflow, TensorFlow, LightGBM, Polars, Superset

Componente ETL y Datamart 1-2

ETL (Extraccion, Transformacion, Carga)

- **51 archivos CSV:** Cabecera de prestamos, amortizaciones, juicios
- Carga masiva a PostgreSQL con `COPY` (2015-2026)
- Proceso incremental con `ON CONFLICT DO UPDATE`

Componente ETL y Datamart 2-2

Datamart (Esquema Estrella)

Dimension	Contenido
dim_tiempo	Mes, anio, trimestre, nombre_mes
dim_riesgo	Codigo y descripcion de riesgo
dim_sector	Codigo y descripcion de sector
dim_sucursal	Sucursal y provincia
fact_creditos_mensual	27 metricas por bloque
fact_predicciones_mensual	Predicciones multi-horizonte

Analisis EVA (Evaluacion de Variables Analiticas)

Modulo Python Reutilizable

- `pipeline.py` - Orquestador EVA (modo notebook/MLflow)
- `analisis_riguroso.py` - Motor de analisis estadistico

Metricas Calculadas

Metrica	Uso
Pearson, Spearman	Correlacion lineal y monotonica
VIF	Multicolinealidad
Chi-cuadrado	Asociacion categorica
Cohen's d, t-test	Diferencia entre grupos
Shapiro-Wilk, KS	Normalidad

Dashboard 3x3

- Heatmap de correlaciones, Top correlaciones
- Valores faltantes, Distribucion target
- Boxplots de mora, Tendencia temporal
- VIF, Recomendaciones automaticas (INCLUIR/EXCLUIR)

EDA Optimizado (EDA001.ipynb)

Problema

La consulta SQL genera **6M+ registros** que agotan la memoria RAM.

Solucion Implementada

- **Polars Lazy Mode:** `pl.read_database()` + `.lazy()` difiere ejecución
- **Procesamiento por año:** Extrae y agrega datos un año a la vez (2015-2025)
- **Agregación incremental:** Solo materializa registros agregados (~miles)

```
for anio in range(2005, 2026):  
    lf = pl.read_database(query=..., connection=engine).lazy()  
    lf = lf.filter(pl.col('fecha_credito').is_not_null())  
    lf = lf.with_columns(pl.col('fecha_credito').cast(pl.Date))  
    df_anio = lf.group_by([...]).agg([...]).collect()  
    fragmentos.append(df_anio)
```

Resultado: Mismo CSV de salida, sin errores de memoria.

Modelos de Inteligencia Artificial

CNN Multi-Horizonte

- **Arquitectura:** Conv1D + BatchNorm + Dropout + Dense
- **Salidas:** 18 neuronas sigmoid (h1 a h18 meses)
- **Entrada:** Ventana de 6 meses, 16 features
- **Target:** `crisis_flag` (score heuristico ≥ 4)

LightGBM

- **18 clasificadores independientes** (uno por horizonte)
- Gradient boosting con optimizacion bayesiana
- Feature importance para interpretabilidad

MLflow - Tracking

- Registro de experimentos y metricas
- Versionado de modelos y artefactos
- Comparacion automatica para seleccion del mejor

Metrica	CNN	LightGBM
AUC-ROC	~0.85	~0.87
Precision	~0.82	~0.84
Recall	~0.79	~0.81

Pipeline de Predicción

1. Modelo seleccionado genera predicciones multi-horizonte
2. Predicciones se guardan en `fact_predicciones_mensual`
3. **Humano en el loop:** experto revisa y aprueba/rechaza

Endpoint de Predicción

```
# predict.py - Inferencia batch
for bloque_id in bloques:
    X_seq, ultimo_mes = generar_secuencia(df_bloque)
    y_pred = modelo.predict(X_scaled)
    # Escritura ON CONFLICT DO UPDATE
```

Dashboard (Superset)

- Visualización por dimensiones: tiempo, riesgo, sector, sucursal
- Filtros interactivos para análisis exploratorio

Estado Actual del Proyecto

Completado

Componente	Estado
ETL (carga CSV a PostgreSQL)	Funcional
Datamart SQL (esquema estrella)	Funcional
Modulo EVA (analisis estadistico)	Funcional
EDA001.ipynb (optimizado Polars)	Funcional
Entrenamiento CNN	Entrenado
Entrenamiento LightGBM	Entrenado
Infraestructura Docker	Desplegada

En Desarrollo

- DAGs Airflow (pasos placeholder)
- Endpoint FastAPI
- Comparacion automatizada de modelos
- Dashboards Superset

Cronograma

Julio 2026		Integracion Airflow + FastAPI
Agosto 2026		Dashboards Superset + Testing
Septiembre 2026		Validacion + Documentacion
Octubre 2026		Sustentacion

Proximos Pasos

1. Integrar scripts reales como tareas en DAGs Airflow
2. Implementar FastAPI endpoint para predicciones
3. Configurar dashboards en Apache Superset
4. Pruebas de integracion end-to-end
5. Documentacion tecnica y de usuario

Preguntas

Omar Velez

omar.velez@yachaytech.edu.ec

Universidad Yachay Tech

Maestria en Inteligencia Artificial

Julio 2026

